

Boundary Case: Cell Timeout / Long-Running Operation as Standalone Architectural Treatment — Formalizing How CKS Handles Cell Executions That Exceed Time or Resource Budgets, Treating Timeouts as A2.62 Bounded Non-Determinism with Rule-Specified Handling per A2.04 and Partial-Completion Provenance per A2.40

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the cell timeout / long-running operation boundary case as a standalone architectural treatment, naming what the architecture commits to when a cell execution exceeds its time or resource budget, what the handling pattern looks like, and what failure modes a deployment must avoid to preserve the determinism, provenance, and replay properties the source paper guarantees.

Abstract

Cell executions in CKS deployments do not always complete within their expected time or resource envelope. A cell may exceed a time budget, exceed a memory or resource budget, hang on an external operation, or otherwise run longer than acceptable. Such timeouts are not architecturally exceptional — they are an ordinary operating condition for long-running cells. The boundary is non-obvious because timeouts may interrupt a cell mid-write: partial substrate state changes may have been committed before the timeout point while the cell's full intended behavior was not realized. The architectural treatment must capture this partial completion without losing the determinism contract (A1.10), cell-behavior determinism (A2.58), bounded non-determinism (A2.62), rule authoring (A2.04, A2.20), retraceability (A1.07, A2.40), or reproducibility (A4.06). This note formalizes the treatment: the timeout is a recorded A2.62 input; the rule per A2.04 specifies handling; partial writes carry full A2.40 six-field provenance; cell-execution-id (A2.40 field 6) distinguishes the timed-out execution from any retry; rollback, if specified, is itself a recorded substrate change; the cell's behavior is deterministic relative to substrate state plus rule plus recorded timeout outcome. The note names the canonical anti-patterns that violate the architecture and the limits of the treatment.

1. Why this boundary needs to be formalized as standalone

The source paper's determinism, provenance, and replay commitments are stated for cells that complete — that read inputs, execute under rules, and write outputs as a single atomic-feeling unit. Real cells do

not always complete. Long-running cells encounter timeouts as an ordinary fact of operation: a data-processing cell may exceed its hour budget; an integration cell may hang on a downstream service; a multi-step orchestration may exhaust wall-clock allocation. The cell's full intended state change does not occur. What does occur — partially.

Two readings are possible. The first treats timeout as a failure to be hidden: the cell is silently cancelled, partial state rolled back automatically, and downstream observers see only the eventual retry. The second treats timeout as a recorded substrate event: partial completion captured with provenance, the rule's specified handling itself recorded, and replay reproducing both. Only the second preserves A1.10, A2.58, and A1.07. Standalone formalization prevents drift into the first by default.

This is the tenth note in Phase A6. The load-bearing source-paper sections are §3.3 (architectural and temporal qualifiers), §4.4 (the mediator role within the orchestration layer), §5 (path retraceability and conflict), and §11.3 (substrate as source of truth).

2. The boundary case scenario and what makes it non-obvious

The scenario is precisely this. A cell begins executing under its orchestration rule, performs some portion of its intended work — possibly including substrate writes, LLM consultations, or external operations — and then exceeds an applicable budget: a wall-clock limit, CPU time, memory, an external-call timeout, or any other resource bound under which it was authorized. The cell does not complete. The deployment must continue.

What makes this non-obvious is the partial-completion middle ground. A cell that fails before any substrate write resembles one that never started: substrate is unchanged, retry behaves as fresh execution. A cell that completes is the ordinary case. A cell that has already committed some substrate writes when the timeout fires is neither: substrate has changed; the cell's intended state change has not occurred; the rule's intended behavior was conditional on completion that did not happen.

The architectural significance: every commitment the source paper makes — substrate as source of truth (§11.3), retraceability (A1.07), determinism given recorded inputs (A1.10, A2.58), reproducibility under replay (A4.06) — must hold for partial completions, not only clean ones. If the architecture handles only clean cases, the boundary is a hole through which determinism, provenance, and replay all leak.

3. Which architectural commitments are stressed

Five commitments are stressed by the cell timeout boundary.

A2.58 cell-behavior determinism. Partial completion means the cell did not produce its full intended state change. Determinism cannot mean "given the same inputs, the cell produces the same complete output." It must mean "given the same inputs and the same recorded timeout outcome, the cell produces the same partial state plus the same handling action." The recorded timeout becomes part of the input set.

A2.62 bounded non-determinism. Timeout events are themselves a non-determinism category. When a timeout fires depends on factors outside the cell's deterministic input set: scheduler decisions,

infrastructure load, external-service latency. A2.62's commitment is that such non-determinism is captured in recorded inputs so replay reproduces.

A2.04 rule authoring. The rule must specify what happens on timeout. A rule that does not is incomplete: behavior on timeout becomes implementation-dependent rather than human-authored. A2.20's Property B — that LLM writes occur under orchestration rules — extends to timeout-handling writes.

A1.07 path retraceability. Partial completions must be recorded with provenance distinguishing them from complete executions. The six-field provenance (A2.40) — when, what, who, why-rule, why-input, cell-execution-id — captures the timed-out execution as a first-class substrate event. Cell-execution-id (A2.40 field 6) distinguishes the timed-out execution from any subsequent retry.

A4.06 reproducibility. Replay must reproduce timeout outcomes. If a cell timed out at minute 47 of a 60-minute budget, replay must see the timeout at minute 47, must record the same partial completion, and must apply the same rule-specified handling. Replay determinism breaks if any of these is left to the runtime.

4. The architectural treatment

The treatment has six elements, each derivable from the load-bearing commitments.

Timeout is a recorded A2.62 input. When a timeout fires, the substrate records a timeout event with full A2.40 six-field provenance: when the timeout occurred, what state the cell was in, who initiated the timeout, the rule, the inputs, and the cell-execution-id. The recorded event is itself substrate state, addressable per A2.59.

The rule per A2.04 specifies the handling. The authorizing rule must include — as substrate-resident authoritative content per A2.46 — a specification of timeout behavior, naming the handling pattern: retry from last completion point, retry from the beginning, queue for human attention, rollback partial writes, fail with a clear error indicating partial completion, escalate to a composition partner, or another pattern the deployment defines. The specification is part of the rule, governed under A1.01's three rights.

Partial substrate writes are recorded with full A2.40 provenance. Writes completed before the timeout are not erased and not unrecorded. They carry the same six-field provenance any complete-execution write carries and are addressable per A2.59. The cell-execution-id ties them to the specific execution that produced them; a retry produces a different cell-execution-id, and its writes are linkable to the retry rather than confused with the original.

Cell-execution-id is the disambiguator. A2.40 field 6 distinguishes a timed-out execution from any retry, rerun, or replay over the same inputs. Without it, partial writes from a timed-out execution and writes from a retry would be indistinguishable, and the rule's specified handling could not be reconstructed from the record.

Rollback is itself a recorded substrate change. Rollback is not silent erasure. It is a recorded substrate change with its own A2.40 provenance — at some recorded time, under some recorded rule,

the partial writes from cell-execution-id X were rolled back. The original partial writes remain in the record (typically as superseded state); the rollback is the supersession event. This preserves A2.59 addressability and A4.06 replay determinism.

Cell behavior is deterministic relative to recorded inputs. Given substrate state at execution start, the rule, and the recorded timeout event with its handling action, the cell's behavior — both what it wrote before timeout and what handling occurred after — is deterministic. Replay produces the same partial writes plus the same handling. The recorded timeout becomes part of the input set, and A2.58 determinism is computed relative to that augmented set.

5. Anti-pattern treatments that would violate the architecture

Eight anti-patterns would, if adopted, violate the commitments stressed in §3.

Cell-timeout-causes-substrate-corruption. Partial substrate writes occur but are not recorded with provenance. Substrate state becomes unaddressable per A2.59 — no record of which cell-execution-id produced which writes. The substrate is corrupted in the architectural sense even if every byte is intact.

Auto-retry-without-recording-timeout. The runtime detects the timeout and silently initiates a retry without recording the timeout event. Replay determinism breaks: replay would not know a timeout occurred, would not reconstruct the retry sequence, and would diverge from the original execution.

LLM-mediated-timeout-handling. A non-mediator LLM determines how the timeout is handled — choosing retry, rollback, escalation, or failure — without rule-mediated authority. This is the A3.13 anti-pattern instantiated: the LLM is making a substrate-write decision (the handling is itself a substrate write) outside human-governed rule mediation. A2.20's Property B is violated.

Vendor-managed-timeout-without-substrate-record. Vendor infrastructure handles the timeout under its own policies, and the substrate has no record of the handling. The substrate-as-source-of-truth commitment (§11.3) is violated: the actual handling lives in vendor logs that are not substrate state, not addressable per A2.59, not subject to A1.01's three rights.

Silent cancellation. The cell is cancelled by the runtime without any substrate event recording the cancellation. Partial writes (if any) have provenance, but the cancellation that ended the execution does not. Replay cannot reconstruct why the cell stopped writing.

Auto-rollback-without-record. Partial writes are silently rolled back, with no record that the rollback occurred. A subsequent observer sees a clean pre-execution state and has no way to know an execution was attempted and undone. Provenance, addressability, and replay all break.

Infinite-cell-execution. Cells are authorized to execute without any timeout specification. The runtime has no architecturally-grounded basis for terminating a cell that runs indefinitely. This violates A2.04 rule completeness for cell behavior: the rule must specify what behaviors are admissible, including the temporal envelope.

Timeout-as-success. A timed-out cell is treated as if it had completed successfully, ignoring partial state. Downstream cells operate on an assumed-complete substrate state that is in fact incomplete. The substrate-as-source-of-truth commitment is violated upstream of any consumer that trusts it.

6. Operational implications

Several implications follow directly from §4.

Rules must specify timeout behavior. No cell can be authorized to execute under a rule that lacks a timeout specification. Rule authoring (Moment 1 in A1.01's two-moments framing) must include the temporal envelope and the handling pattern. The cost scales with rule variety, not substrate size or cell-execution count.

Partial writes are normal substrate state. Partial writes from timed-out cells are ordinary substrate content — addressable, queryable, and governed by the same A1.01 rights as any other content. Tooling that hides them as "incomplete" creates a false picture of substrate state.

Rollback is a substrate change. Rollback events are visible in the substrate's record, governed by the same authority architecture, and the substrate's record of what was rolled back remains addressable.

Cell-execution-id is load-bearing. Tooling that aggregates writes by rule, cell type, or inputs without distinguishing cell-execution-id will conflate timed-out and complete executions and break the disambiguation the field was introduced to provide.

Long-running operations require explicit timeout specification. Cell types expected to run for non-trivial duration must have timeout specifications considered as part of rule authoring, not deferred to runtime defaults. Runtime defaults are infrastructure, not rule content; relying on them moves authority off-substrate in violation of §11.3.

7. Limits of the architectural treatment

The treatment in §4 applies specifically to cell-level timeouts. It does not extend automatically to several adjacent boundary cases.

LLM consultation timeouts within cells (A6.04). When an LLM consultation inside an executing cell times out, the treatment is narrower: it concerns the consultation's role in cell execution rather than the cell-execution timeout as a whole. A6.10 does not subsume A6.04; the two compose, with A6.04's per-consultation handling occurring inside A6.10's per-cell envelope.

Network partition affecting cells (A6.08). When a cell is unreachable due to network partition rather than budget exhaustion, the substrate's record of the cell's state may itself be temporarily inaccessible; substrate-as-source-of-truth is being stressed at the substrate layer rather than the cell-behavior layer.

Composition partner failures (A6.05). When a cell's failure to complete is due to a composition partner's failure rather than its own budget exhaustion, the boundary is in the composition contract, not the cell's timeout specification.

Substrate-level capacity issues (A6.07). When the substrate itself cannot accept further writes due to capacity exhaustion, the cell's inability to complete is a substrate-layer concern, not a cell-execution-budget concern.

Infrastructure-level failures. Power loss, hardware failure, and similar events are not architecturally cell timeouts. The architecture's commitment is that recovered substrate state — once the host returns

— must be consistent with the A2.40 provenance recorded before the failure, and A2.59 addressability must hold across recovery.

The treatment in §4 is necessary for the cell-timeout case and does not pretend to be sufficient for the adjacent cases.

8. The architectural test

A deployment instantiates the cell-timeout / long-running-operation architectural treatment if and only if all of the following are true at all times:

1. Every cell type's authorizing rule includes a timeout specification, identifying both the temporal envelope and the handling pattern.
2. When a cell exceeds its budget, the timeout event is recorded as substrate state with full A2.40 six-field provenance.
3. Any substrate writes the cell committed before the timeout point are recorded with provenance that includes the cell-execution-id of the timed-out execution.
4. The handling action specified by the rule (retry, rollback, queue, fail, escalate, or other) is itself executed under rule authority and its execution is recorded as substrate state.
5. If rollback is the specified handling, the rollback is a recorded substrate change preserving the original partial writes' addressability.
6. Replay of the substrate state, given the recorded timeout events as inputs, reproduces the same partial completions and the same handling actions.
7. No LLM, no vendor system, and no infrastructure layer determines timeout handling outside the rule's authority.

A deployment that fails any of (1)–(7) may continue to operate, and may even handle most timeouts acceptably in practice, but does not instantiate the architectural treatment in the CKS sense.

9. Why naming this boundary as standalone matters

The boundary is operationally common, and the architectural significance of how the substrate captures timeouts is easy to overlook when designing around the clean-completion case. Without an explicit treatment, a deployment will reach for whatever timeout-handling its host runtime, vendor system, or workflow orchestrator provides, and the resulting handling will live outside substrate provenance, outside rule authority, and outside the commitments the source paper makes.

A deployment that records cell timeouts as substrate state with provenance, specifies handling in rules per A2.04, distinguishes timed-out executions via cell-execution-id, treats rollback as itself a recorded change, and produces replay-reproducible behavior under partial completion is instantiating an architectural pattern. That pattern is now formalized as prior art under the author's name, derivable from the source paper's §3.3, §4.4, §5, and §11.3, and stress-testing the commitments in A1.07, A1.10, A2.04, A2.20, A2.40, A2.55, A2.58, A2.59, A2.62, A4.06, and A4.11. Subsequent Phase A6 notes will cover further boundary cases, including the substrate concurrent-write race (A6.11), multi-author rule

conflict during composition (A6.12), and additional boundaries through A6.15.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: Cell Timeout / Long-Running Operation as Standalone Architectural Treatment — Formalizing How CKS Handles Cell Executions That Exceed Time or Resource Budgets, Treating Timeouts as A2.62 Bounded Non-Determinism with Rule-Specified Handling per A2.04 and Partial-Completion Provenance per A2.40*. May 7, 2026. ORCID: 0009-0004-8065-3235.